

# HEALTH-AI — Clinical ML Visualisation Tool

2-Page Project Report | Cankaya University, Software Engineering | Spring 2025-2026

## Team & Project Overview

| Name                   | Student ID | Role                    |
|------------------------|------------|-------------------------|
| Sina Emre Bizlik       | 202128037  | Product Owner           |
| Bilal Celebi           | 202128026  | Lead Developer          |
| Anil Alper Koc         | 202128024  | QA / Documentation Lead |
| Yusuf Batuhan Demirbag | 20228012   | Developer               |
| Cagatay Temel          | 202128034  | UX Designer             |

HEALTH-AI is a full-stack web application that guides medical students and researchers through a complete machine learning pipeline applied to clinical data. The system supports 20 medical domains, 6 ML algorithms, real SMOTE balancing, SHAP-style explainability, fairness/bias analysis, and automated PDF certificate generation. All trained models persist across backend restarts via joblib serialisation and SQLite.

## System Architecture

| Layer       | Technology                            | Responsibility                                       |
|-------------|---------------------------------------|------------------------------------------------------|
| Frontend    | React 18 + Vite + Tailwind + Recharts | 7-step UI, charts, dark/light mode                   |
| API Gateway | FastAPI (Python 3.11) + Uvicorn       | REST endpoints, request validation (Pydantic)        |
| ML Engine   | scikit-learn + imbalanced-learn       | Pipeline: Imputer -> Scaler -> (SMOTE) -> Classifier |
| Persistence | joblib + SQLite                       | Model .pkl files, training history, CSV uploads      |
| PDF Engine  | ReportLab                             | Certificate generation with unique run ID            |
| Container   | Docker + Docker Compose + nginx       | Single-command deployment, nginx reverse proxy       |

## Key Architectural Decisions

### D: sklearn Pipeline for zero data leakage

All preprocessing steps (imputation, scaling, SMOTE) are encapsulated inside a single sklearn Pipeline object. This guarantees the scaler is fitted only on `X_train` and never sees `X_test` — a critical correctness requirement for medical ML systems.

### D: joblib + active\_data.pkl for full persistence

Beyond saving the Pipeline weights, we also persist `X_test/X_train/y_test/y_train` arrays. This means that after a backend restart, all Step 5-7 endpoints (waterfall, subgroup analysis, patients) continue to work without re-training.

### D: Single-file FastAPI backend

All ML logic, API endpoints, and persistence utilities live in one 1 024-line `main.py`. This simplifies Docker deployment and keeps the learning curve low for a university project.

### D: Synthetic sample data per domain

# HEALTH-AI — Clinical ML Visualisation Tool

2-Page Project Report | Cankaya University, Software Engineering | Spring 2025-2026

Each domain generates realistic-looking synthetic data using a multivariate normal distribution with domain-appropriate feature names. Users can explore the full pipeline without needing real patient data.

# HEALTH-AI — Clinical ML Visualisation Tool

2-Page Project Report | Cankaya University, Software Engineering | Spring 2025-2026

## Sprint Timeline & Delivery

| Sprint   | Period               | Key Deliverables                                                                     | Status |
|----------|----------------------|--------------------------------------------------------------------------------------|--------|
| Sprint 1 | 19 Feb – 4 Mar 2026  | Project setup, Docker skeleton, domain selection UI, FastAPI scaffold                | Done   |
| Sprint 2 | 4 Mar – 18 Mar 2026  | CSV upload, column mapping, data preparation screen, normalization                   | Done   |
| Sprint 3 | 18 Mar – 1 Apr 2026  | Model training (6 algorithms), hyperparameter UI, KNN canvas, metrics dashboard      | Done   |
| Sprint 4 | 1 Apr – 15 Apr 2026  | Feature importance, waterfall chart, subgroup bias analysis, joblib persistence      | Done   |
| Sprint 5 | 15 Apr – 29 Apr 2026 | Dark mode, domain palettes, animated training progress, PDF certification, bug fixes | Done   |

## Challenges & How We Solved Them

### C: Data Leakage (Critical ML bug)

Initial implementation called `scaler.fit_transform()` on the entire dataset before the train/test split. This allowed test-set statistics to influence the scaler, artificially inflating performance metrics. Solution: moved all preprocessing into a sklearn Pipeline, guaranteeing that `fit()` is called only on `X_train`.

### C: Target Column with String Labels

The system crashed when the target column contained string values such as 'Yes'/'No' or 'Diabetic'/'Non-Diabetic'. Solution: applied `LabelEncoder` before the train/test split, transparently converting any string label to integer classes.

### C: SMOTE Applied to Full Dataset

The original oversampling applied Gaussian noise to the full dataset, which is both statistically incorrect and not real SMOTE. Solution: integrated `imbalanced-learn`'s SMOTE into the Pipeline so it executes only on the training split.

### C: State Loss on Backend Restart

The waterfall and subgroup endpoints relied on `X_test/y_test` stored only in RAM. A backend restart lost these arrays, causing HTTP 400 errors on Steps 6-7. Solution: persisted the arrays to `active_data.pkl` via `joblib`, restored on startup.

### C: White Screen on Step 4

The KNN parameter panel crashed the entire React tree because `SParam` and `DropParam` components were referenced but never defined in the file. Solution: defined both components at the top of `Step4Model.jsx` with full `PropTypes`.

## Lessons Learned

### L: Always build the ML pipeline end-to-end first, then add UI.

We spent Sprint 2 on frontend while the backend had critical data-leakage bugs that were only discovered in Sprint 4. A working prototype first approach would have caught this earlier.

### L: Persistence must be designed from day one.

Adding `joblib` + `SQLite` persistence in Sprint 4 required refactoring the entire training flow. Had we designed for restart-resilience from Sprint 1, the implementation would have been far simpler.

### L: Component errors in React kill the whole page.

# HEALTH-AI — Clinical ML Visualisation Tool

2-Page Project Report | Cankaya University, Software Engineering | Spring 2025-2026

An undefined component reference in Step 4 produced a silent white screen with no useful error message in production. We now use React Error Boundaries and check every component reference before committing.

## **L: Synthetic data must be domain-realistic.**

Using the same random seed and covariance matrix for all 20 domains produced identical accuracy scores, which looked suspicious. In a future version, each domain should have a calibrated synthetic generator.

## **Conclusion**

---

HEALTH-AI successfully delivers a complete, functional clinical ML pipeline in a polished web interface. All seven steps are operational, the system correctly avoids data leakage, uses real SMOTE oversampling, persists models across restarts, and provides SHAP-style explainability and fairness analysis aligned with EU AI Act principles. The five-member team completed five sprints on schedule, producing a system that can be demonstrated live with `docker-compose up --build`.